

Fondamenti di Analisi dei Dati

from **data analysis** to **predictive techniques**

Prof. Antonino Furnari (antonino.furnari@unict.it)

Corso di Studi in Informatica

Dip. di Matematica e Informatica

Università di Catania



Università
di Catania

Generative Classifiers and Naive Bayes

Generative classification models, and the pragmatic power of a conditional independence assumption

Two Approaches to Classification

Discriminative Models

Models like Logistic Regression focus on learning the **decision boundary** between classes. They directly model $P(Y = k|X = \mathbf{x})$.

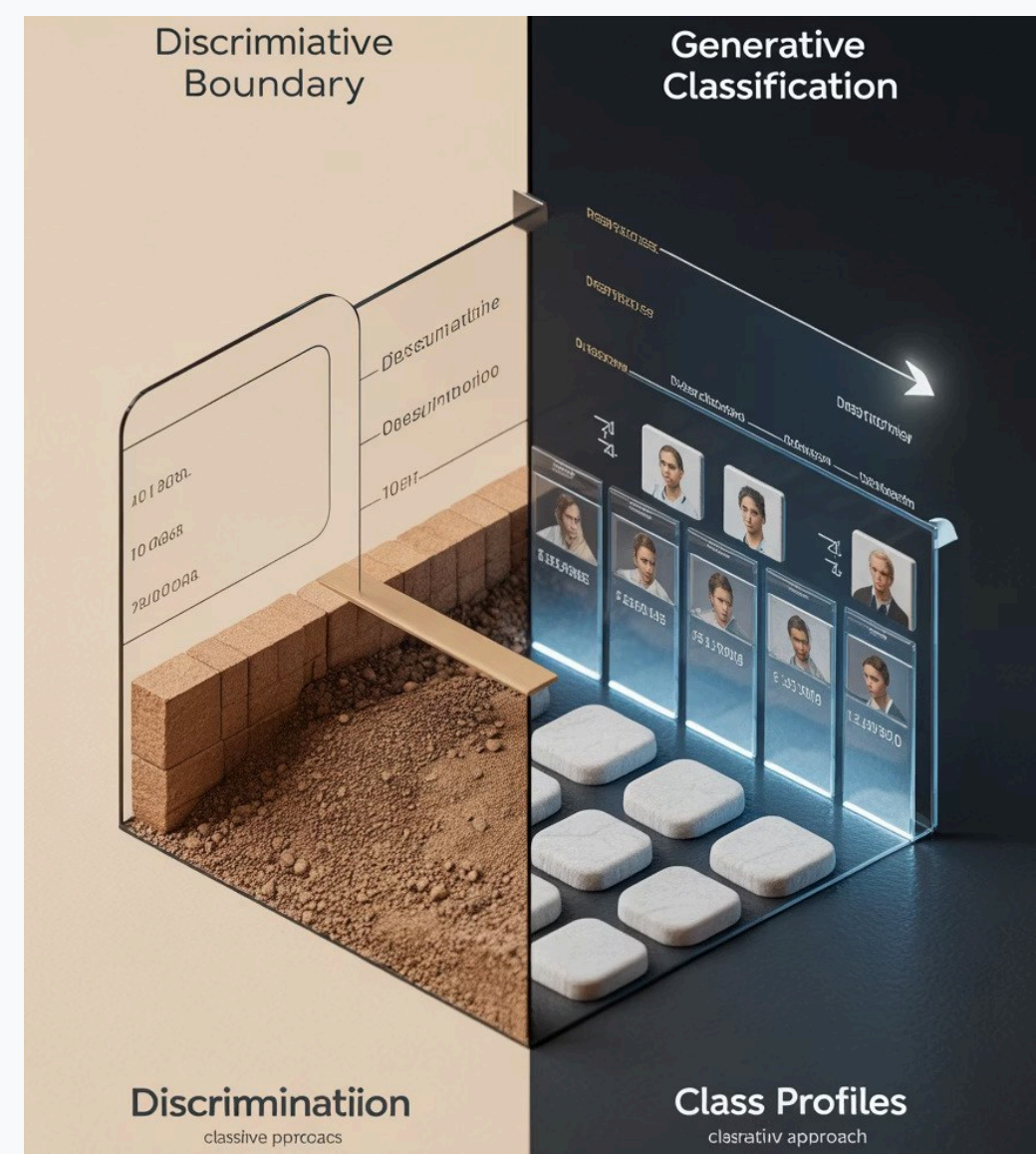
Goal: find the line (or surface) that best separates the classes.

Throughout this lecture Y is discrete and finite — our classes — while X may be continuous or discrete, scalar or multidimensional.

Generative Models

Models like Naive Bayes learn the **"story" or "profile"** of each class independently. They model the joint probability $P(X, Y)$.

Goal: understand what each class looks like, then classify based on similarity.



The Generative Approach

Recall that:

$$P(X, Y) = P(X|Y)P(Y)$$

Generative models build a complete probabilistic picture by modelling two components separately:

The Likelihood

$$P(X = \mathbf{x} | Y = k)$$

"What does a typical X look like *for this class*?"

The Prior

$$P(Y = k)$$

"How common is *this class* in general?"

Once these two components are learned, we can use them together for classification through Bayes' theorem.



Maximum A Posteriori (MAP)

The MAP principle finds the most probable class after observing the data:

$$h(\mathbf{x}) = \arg \max_k P(Y = k | X = \mathbf{x})$$

Using Bayes' theorem:

$$h(\mathbf{x}) = \arg \max_k \frac{P(X = \mathbf{x} | Y = k) P(Y = k)}{P(X = \mathbf{x})}$$

Since $P(X = \mathbf{x})$ does not depend on the class k , it cannot change which class wins the maximisation, and we can drop it:

$$P(Y = k | \mathbf{x}) \propto P(X | Y = k) P(Y = k)$$

This leaves the MAP classification rule:

$$h(\mathbf{x}) = \arg \max_k \underbrace{P(X | Y = k)}_{\text{Likelihood}} \underbrace{P(Y = k)}_{\text{Prior}}$$

- This elegant formulation shows that classification reduces to computing two quantities: how likely the data is for each class (the likelihood) and how common each class is (the prior).



Estimating the Prior $P(Y)$

The prior probability represents how common each class is. If observing a class $Y = k$ is not very common, then $P(Y = k)$ is small. We have several approaches to estimate it:

01

Dataset Proportions

Count the examples in the training data. With 800 non-spam and 200 spam e-mails, and `Class 0 = Spam`, `Class 1 = Non Spam`, we take $P(Y = 0) = 0.2$ and $P(Y = 1) = 0.8$.

02

Real-World Statistics

Study the actual proportions in the world: ask a large sample of people how many e-mails they receive and how many of those are spam.

03

Uniform Assumption

When we lack information on the phenomenon, assume all classes are equally probable: $P(Y = k) = \frac{1}{m}$, where m is the number of classes.

The prior should be read in Bayesian terms: by choosing it we are stating our **degree of belief** about which classes are more or less likely in the system, before seeing the data.





The Likelihood Challenge

While estimating the prior is straightforward, computing the likelihood $P(X|Y = k)$ is the central challenge of generative modelling. The approach depends on the type and the dimensionality of the features.

For M different classes, a general recipe is to group all the observations of a class $Y = k$ — call X_k the random variable of that group — and estimate $P(X_k)$:

$$P(X = x|Y = k) = P(X_k)$$

Repeating this for every class gives us $P(X|Y = k)$. To estimate $P(X_k)$ we will need assumptions, and the assumptions we make determine which generative model we get.

The entire challenge of generative modelling lies in estimating the likelihood. This is where we run (again) into the Curse of Dimensionality.

From Ideal to Practical

The MAP rule is simple; the likelihood is not. The rest of this lecture is a single chain of simplifications, each one buying tractability at the price of flexibility:

01

Contingency Table

The ideal model for **discrete** features. Exact, and impossible past a handful of features.

03

LDA

One covariance matrix, **shared** by all classes. Fewer parameters, linear boundaries.

02

QDA

The ideal model for **continuous** features. One full covariance matrix per class.

04

Naive Bayes

Features **independent given the class**. One univariate distribution per feature.

Each step removes parameters. QDA estimates roughly $K \times \frac{d^2}{2}$ covariance parameters, LDA roughly $\frac{d^2}{2}$, Gaussian naive Bayes only $K \times d$ variances. For text, with $d = 10\,000$, only the last one is even computable.

- Which simplification is acceptable is not a mathematical question but an empirical one: it depends on how much data we have and on how strongly the features are correlated within a class.



Guiding Example: Spam Detector

Spam detection is a classic and highly illustrative example of generative classification. Our objective is to classify an incoming e-mail as either "spam" or "ham" (not spam) based on its characteristics.

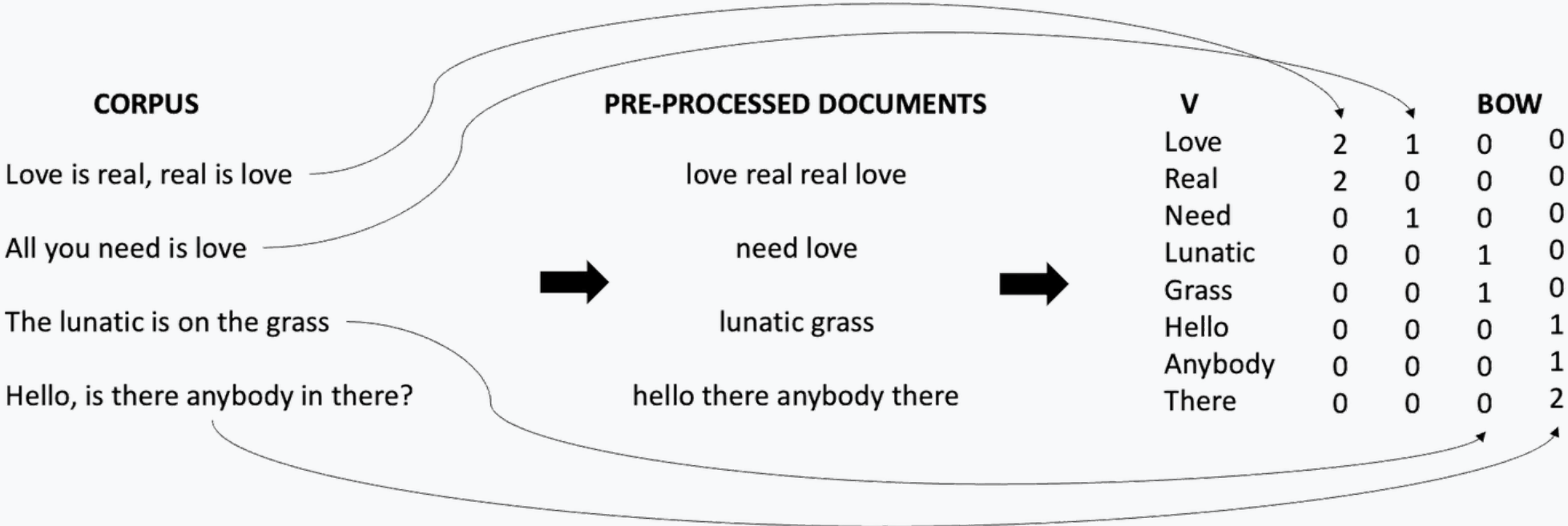
The problem is interesting precisely because of its inherent **high dimensionality**. Every e-mail can be described by an enormous number of potential features, creating a vast feature space.

Our Features: Word Frequencies

The presence or the count of every word in a potentially massive vocabulary becomes a feature. With a vocabulary of 10k words, we have 10k dimensions.
A feature vector can be very sparse, counting only the words actually appearing in the e-mail.

Each of these elements contributes to a high-dimensional vector, which makes estimating the likelihood $P(X|Y = k)$ complex without strategic simplifications.

The figure below shows how a text is converted into a vector of word counts. In this process we usually remove the less meaningful words, such as articles and prepositions, which are called "stop words":



The "Ideal" Model for Discrete Data (and its Limitations)

If our features are discrete (e.g. `Offer=Yes`, `Free=No`), we could try to model $P(X|Y)$ by building a **giant contingency table** for every single combination of features.

The Conceptual Approach

This "ideal" model would meticulously track the probability of every specific feature combination for each class, providing a complete picture of $P(X|Y)$.

The Problem: Curse of Dimensionality

With 10,000 words (features) in our vocabulary, we would need to estimate the probability of $2^{10,000}$ possible combinations. This is computationally and statistically impossible, and the data is extremely sparse.

This massive number highlights why modelling $P(X|Y)$ directly, for high-dimensional discrete data, is unfeasible without significant simplifications.

- The approach does work in simple cases, with few features and few levels. It just does not generalise to the cases we care about.



The "Ideal" Model for Continuous Data: QDA

For continuous features we cannot use contingency tables. The most flexible "ideal" approach is to model $P(X = \mathbf{x} | Y = k)$ as a **Multivariate Gaussian (Normal) Distribution**:

$$P(X = x | Y = k) = N(\mu_k, \Sigma_k)$$

In practice we compute a mean vector and a covariance matrix for each class, from the subset of the data belonging to that class. This is the foundation of **Quadratic Discriminant Analysis (QDA)**.

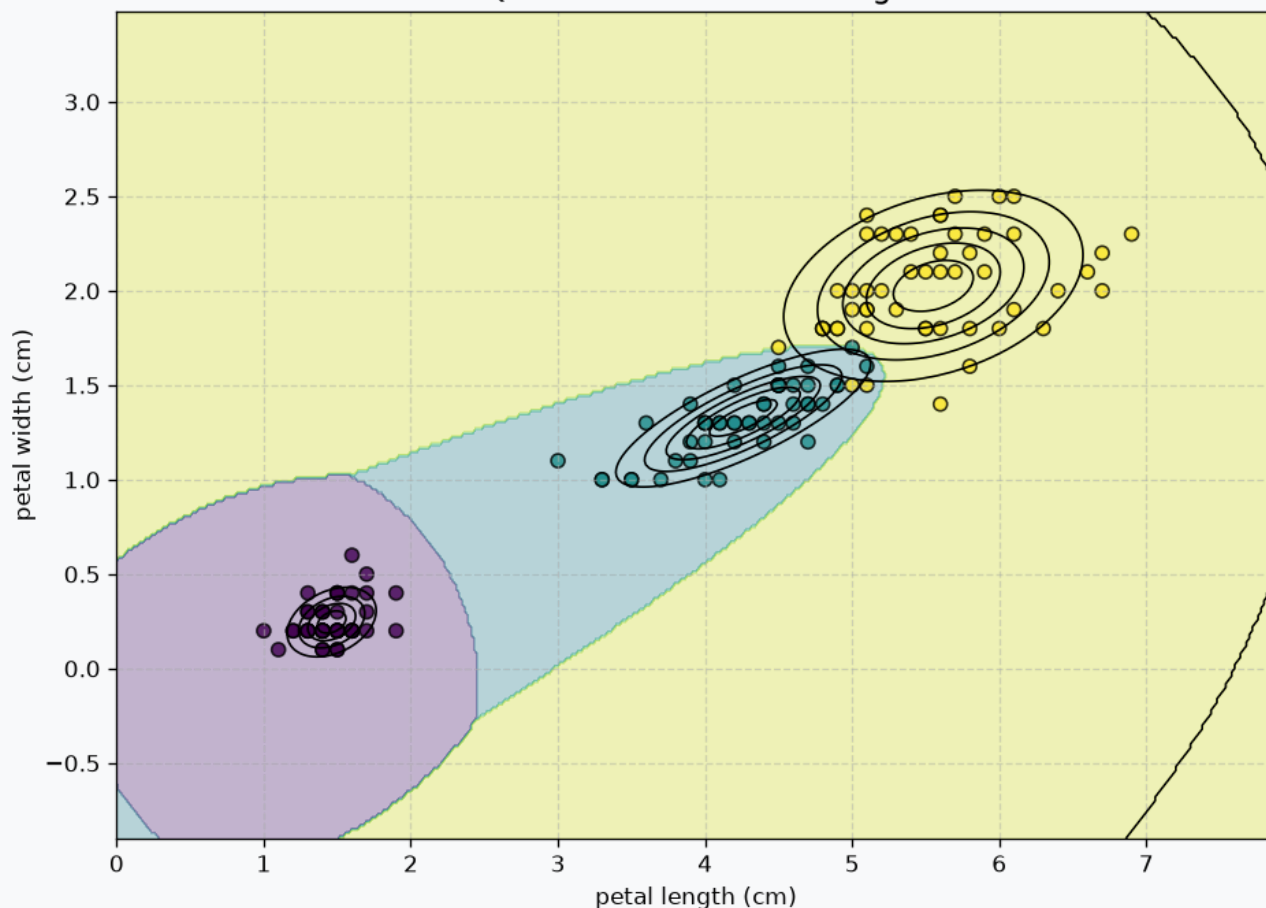
Assumptions

Each class k has its own mean vector μ_k and its own, *full* covariance matrix Σ_k .

The Computational Problem

Estimating a full covariance matrix Σ_k for d features requires about $\frac{d^2}{2}$ parameters **per class**. For 100 features that is $\approx 5,000$ parameters per class: computationally intensive, and prone to overfitting when a class has little data.

QDA with Gaussian Fittings



The Resulting Boundary

QDA produces a powerful, **curved (quadratic)** decision boundary between classes, reflecting its flexibility in modelling distinct class distributions.

When we have enough data QDA can lead to good results, but it can easily overfit when we don't.

Note that each class has its own Gaussian distribution, and that the distributions can be very different from each other.

The First Simplification: Linear Discriminant Analysis (LDA)

The computational limitations of QDA lead us to our first compromise: **Linear Discriminant Analysis (LDA)**, which simplifies the model for greater stability and practicality.

LDA still models $P(X = \mathbf{x}|Y = k)$ as a Multivariate Gaussian:

$$P(X = x|Y = k) = N(\mu_k, \Sigma)$$

The means μ_k still differ, but the covariance matrix Σ is now assumed to be the same for all classes. This reduces the parameters of the covariance from about $K \times \frac{d^2}{2}$ to about $\frac{d^2}{2}$.

01

Assumption: Shared Covariance

All classes *share* the same covariance matrix, $\Sigma_k = \Sigma$ for all k . A significant reduction in model complexity.

02

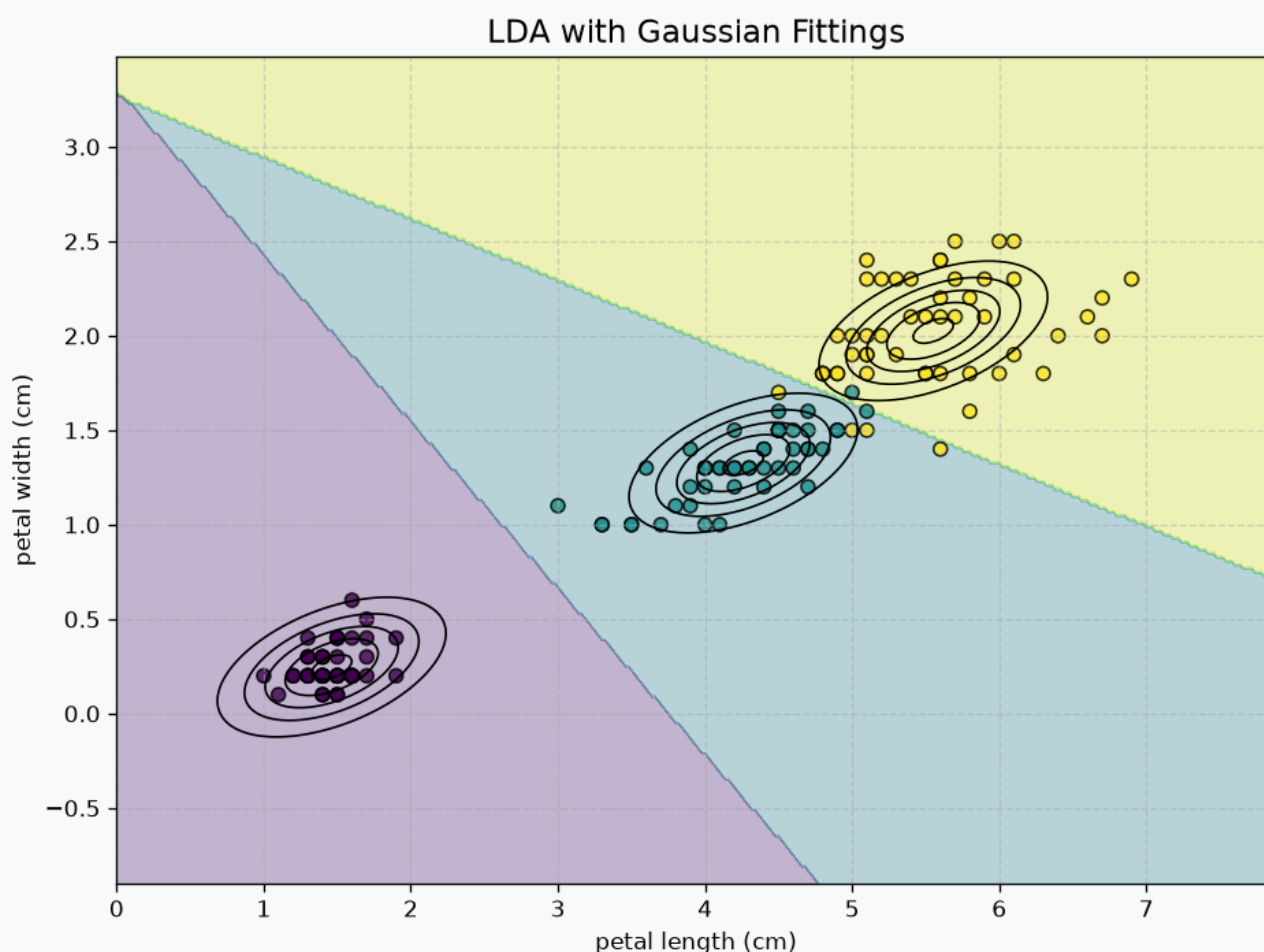
The Benefit: Stability

We estimate just *one* full Σ , using all the data rather than class-grouped data. Much more stable when data is limited.

03

The "Nightmare" (Still)

We still need one full $\frac{d^2}{2}$ matrix. For text, with $d = 10,000$, this remains computationally and statistically impossible.



The Resulting Boundary

The shared covariance forces the mathematics to simplify, and the decision boundary is always **linear**.

Every class now has the same "shape": less flexible than QDA, but more robust and less prone to overfitting when data is scarce or dimensionality is high.

The Naive Assumption

Naive Bayes makes one powerful, simplifying assumption that solves the dimensionality problem. Start from the general factorisation of the likelihood for $X = (X_1, \dots, X_n)$:

$$P(X_1, \dots, X_n | Y) = P(X_1 | Y) P(X_2 | X_1, Y) \dots P(X_n | X_1, \dots, X_{n-1}, Y)$$

This is not a helpful factorisation: every term is conditioned on the other features, and none of them is easy to model. So we assume instead:

All features X_i are **conditionally independent** given the class $Y = k$.

$$X_i \perp X_j \mid Y, \forall i \neq j$$

Since $X \perp Y | Z \Leftrightarrow P(X, Y | Z) = P(X | Z)P(Y | Z)$, this leads to:

$$P(X_1, \dots, X_n | Y) = P(X_1 | Y) P(X_2 | Y) \dots P(X_n | Y)$$

Instead of modelling one complex joint distribution, we model n simple univariate distributions.

- The assumption is called "naive" because it is often violated in practice. Surprisingly, Naive Bayes works remarkably well despite the violation.



Why the Assumption is "Naive"

Spam classification example: consider word counts in e-mails. The naive assumption claims that **within non-spam e-mails, the count of one word does not influence the count of another word.**

Vacation E-mail

Words like "trip", "flight" and "luggage" appear together frequently.

Work E-mail

Words like "meeting", "report" and "deadline" co-occur often.

Clearly, word counts are **not independent** — they are correlated by topic, and both e-mails above are legitimate, so the correlation lives *inside* one class. Yet Naive Bayes still performs excellently in practice, because the assumption simplifies the computation dramatically while preserving enough signal for accurate classification.



The Simplified MAP Rule

With the conditional independence assumption, our classification rule becomes simple:

$$f(\mathbf{x}) = \arg \max_k P(\mathbf{x}_1|Y = k)P(\mathbf{x}_2|Y = k) \dots P(\mathbf{x}_n|Y = k)P(Y = k)$$

Each $P(X_i|Y = k)$ term is now easy to model, since X_i is one-dimensional. We choose the type of distribution based on our data:

Gaussian Naive Bayes

For **continuous** features, model each $P(X_i|Y)$ as a univariate Gaussian distribution.

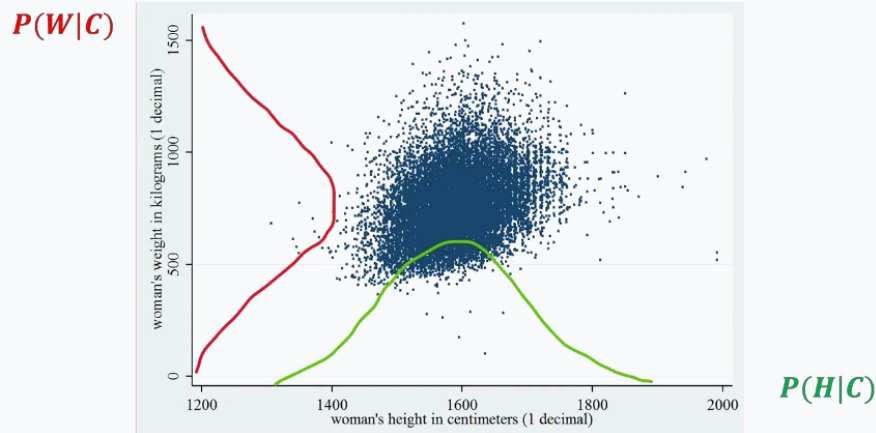
Multinomial Naive Bayes

For **discrete count** data, such as text, model the counts with multinomial distributions.

These are the two flavours of Naive Bayes we will study; the name of the method is simply the name of the distribution used for the single terms.



Gaussian Naive Bayes in Action



Consider classifying biological sex from height and weight, $X = [H, W]$. For each feature and each class we estimate a simple 1D Gaussian, from four samples — the heights and the weights of the subjects of each class:

- $P(H = h|C = 0) = N(x; \mu_1, \sigma_1)$
- $P(W = w|C = 0) = N(x; \mu_2, \sigma_2)$
- $P(H = h|C = 1) = N(x; \mu_3, \sigma_3)$
- $P(W = w|C = 1) = N(x; \mu_4, \sigma_4)$

The plot on the left shows the estimation of the height and weight distributions for one of the two classes. The same computation is done for the other.

Classification rule: assign (h, w) to class 1 if

$$P(h|C = 1)P(w|C = 1)P(C = 1) > P(h|C = 0)P(w|C = 0)P(C = 0)$$

and to class 0 otherwise.

Implications of the Naive Assumption

The conditional independence assumption is a strong one, but it offers crucial benefits and imposes specific characteristics on the model:

Diagonal Covariance Matrix

Given the class, assuming that the features X_i are independent implies that the off-diagonal elements of the covariance matrix Σ_k are zero:

$$\Sigma_k = \begin{pmatrix} \sigma_{k1}^2 & 0 & \dots & 0 \\ 0 & \sigma_{k2}^2 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & \sigma_{kn}^2 \end{pmatrix}$$

Non-Linear Decision Boundaries

Each class keeps its *own* diagonal covariance matrix ($\Sigma_k \neq \Sigma_j$). That flexibility lets Gaussian Naive Bayes produce non-linear decision boundaries, similar in concept to QDA but with stricter constraints on the shape of the distributions.

Axis-Aligned Distributions

A diagonal covariance matrix means the multivariate Gaussian of each class is "axis-aligned": its elliptical contours are parallel to the coordinate axes. This simplifies computation and estimation, and prevents the model from capturing any rotation, or correlation between features within a class.

No Intra-Class Correlation

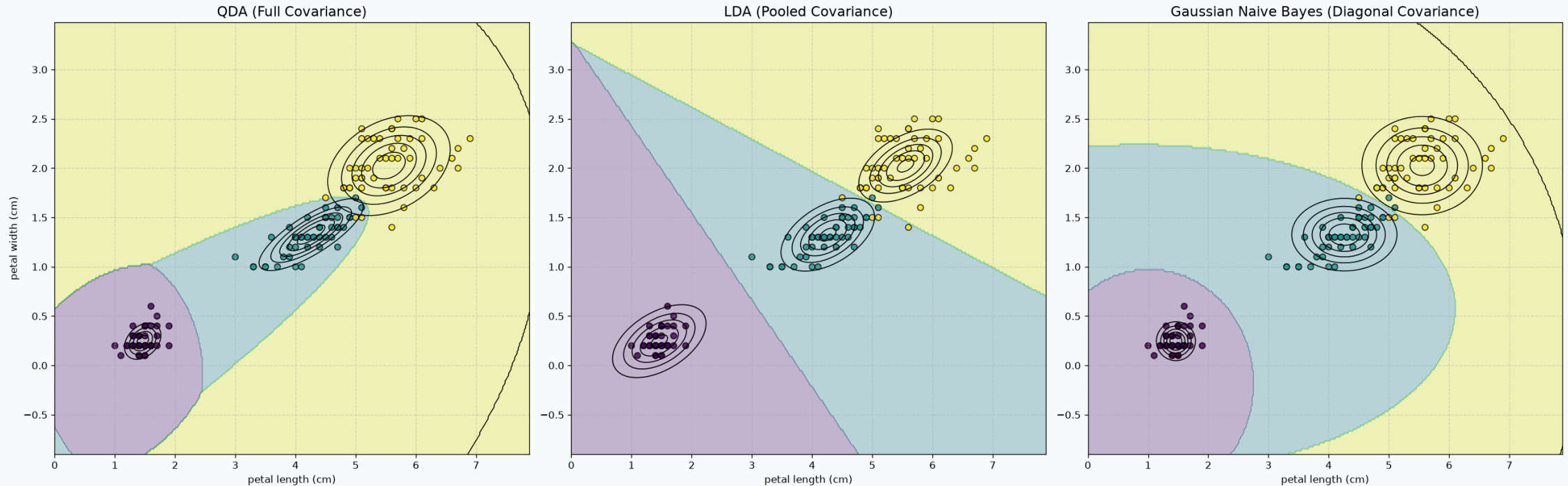
The primary limitation: Naive Bayes explicitly **cannot** model correlations between features **within** the same class. If taller individuals in a class also tend to be heavier, the model cannot capture it — it treats height and weight as independent given the class.

Hence, Naive Bayes may not work well when there is high correlation between features.

This trade-off — simplified modelling for the inability to capture intra-class correlations — is what defines Naive Bayes.



Naive Bayes Decision Boundaries



This figure summarises the three models:

- **QDA (left):** most flexible. It learns a separate, *rotated* ellipse (full Σ_k) for each class. Its boundaries are quadratic.
- **LDA (middle):** the compromise. It learns *one* pooled, *rotated* ellipse (full Σ) and uses it for all classes. Its boundaries are linear.
- **GNB (right):** the simplest. It learns a separate, *axis-aligned* ellipse (diagonal Σ_k) for each class. Its boundaries are quadratic, but cannot capture any feature correlation (tilt).

Multinomial Naive Bayes for Text

For text classification with word counts (the "bag of words" representation), the features are discrete. A document is represented as:

$$\mathbf{x} = (x_1, x_2, x_3, \dots, x_d)$$

where $x_i \in \mathbb{N}$ is the count of word i in the document $\mathbf{x}$ and $d = |V|$ is the size of the vocabulary V . The probability of a document $\mathbf{x}$ belonging to class k is then:

$$P(\mathbf{x}|Y = k) = P(x_1, x_2, \dots, x_d|Y = k)$$

We model this probability with a **multinomial distribution**, treating document generation as repeatedly sampling words from a vocabulary. We estimate one such distribution per class.

- Remember that the multinomial distribution models the probability of obtaining exactly $(x_1, \dots, x_d)$ occurrences of each of the d possible outcomes, in a sequence of $n = \sum_i x_i$ independent experiments which follow categorical distributions with probabilities $p_{k1}, \dots, p_{kd}$. In our case:
 - **individual experiment**: the process of generating a single word;
 - **d possible outcomes**: the words in the vocabulary;
 - **n independent experiments**: the words in a document, $n = \sum_i x_i$.

The experiments are *independent* because of the naive assumption: given a class, word counts are independent.



Fitting the Multinomial Distribution to the Data

Using the analytical form of the multinomial distribution, we can write the likelihood:

$$P(\mathbf{x}|Y = k) = \frac{(\sum_i x_i)!}{x_1! \dots x_d!} \cdot p_{k1}^{x_1} \cdot p_{k2}^{x_2} \cdot \dots \cdot p_{kd}^{x_d}$$

In the expression above:

- $n = \sum_i x_i$ is the total number of experiments (the words in the document);
- x_i is the count of word i in the current document $\mathbf{x}$;
- p_{ki} is the probability of observing word i if we randomly pick a word from a document of class $Y = k$.

In practice, to use the formula above, we need to estimate the parameters p_{ki} , for $i = 1, \dots, |V|$ and $k \in \{1, \dots, M\}$, where $|V|$ is the size of the vocabulary and M the number of classes.

Once we have all these parameters, we obtain our classifier as:

$$h(\mathbf{x}) = \arg \max_k P(Y = k) \cdot p_{k1}^{x_1} \cdot p_{k2}^{x_2} \cdot \dots \cdot p_{kd}^{x_d}$$

where the big factorial term $\frac{(\sum_i x_i)!}{x_1! \dots x_d!}$ is dropped: it depends only on $\mathbf{x}$, not on k , so it cannot change which class wins.



Estimating Word Probabilities

We estimate the probability of word i for class k by counting:

$$p_{ki} = \frac{\text{Total count of word } i \text{ in all documents of class } k}{\text{Total count of all words in all documents of class } k}$$

Using the Iverson bracket notation, where $[y^{(j)} = k]$ is 1 if true and 0 otherwise:

$$p_{ki} = \frac{\sum_j [y^{(j)} = k] x_i^{(j)}}{\sum_{l=1}^d \left(\sum_j [y^{(j)} = k] x_l^{(j)} \right)}$$

- **Numerator:** sums the counts of word i *only* in the documents j of class k .
- **Denominator:** sums the counts of *all* words ($l = 1$ to d) in *all* documents j of class k .

The prior probability is estimated as the fraction of documents belonging to class k :

$$P(Y = k) = \frac{\sum_j [y^{(j)} = k]}{N}$$

where N is the number of observations in the training set. Alternatively, we can assume uniform priors and set $P(Y = k) = \frac{1}{M}$.



Practical "Gotchas" (1): The Zero-Probability Problem

A critical issue arises when we classify a document containing a word that never appeared in a given class in the training data.

The Challenge: Zero Probabilities

Consider a new e-mail containing the word "crypto". If the training data for the 'Spam' class **never** included that word, its estimated probability $p_{k, \text{'crypto'}}$ for that class is 0:

$$h(\mathbf{x}) \propto P(Y = \text{Spam}) \cdot \dots \cdot p_{k, \text{'crypto'}}^{x_{\text{'crypto'}}} \cdot \dots = P(Y = \text{Spam}) \cdot \dots \cdot 0 \cdot \dots = 0$$

Because of this single zero, the entire probability of the 'Spam' class becomes zero, disabling classification for that instance.

The Solution: Laplace (Add-One) Smoothing

We add a "pseudo-count" (typically 1) to each word's count in every class, so that even unobserved words have a non-zero, if small, probability:

$$p_{ki} = \frac{\left(\sum_j [y^{(j)} = k] x_i^{(j)} \right) + 1}{\left(\sum_{l=1}^d \sum_j [y^{(j)} = k] x_l^{(j)} \right) + |V|}$$

- The $+1$ in the numerator means we "pretend" we have seen each word at least once.
- The $+|V|$ in the denominator, where $|V|$ is the vocabulary size, corrects for the added pseudo-counts, so that the probabilities of each class still sum to 1.

In practice, words which were not present in the training corpus for a given class get assigned a uniform probability of $\frac{1}{|V|}$.



Practical "Gotchas" (2): Arithmetic Underflow

Our formulation multiplies small (< 1) probability values $p_{k1}^{x_1} \cdot p_{k2}^{x_2} \cdot \dots \cdot p_{kd}^{x_d}$. This can lead to **arithmetic underflow**: the product becomes so tiny that the computer rounds it to zero, and classification is invalid.

The solution is to use **log-probabilities**. Since the logarithm is monotonic, maximising $\log P(X|Y)$ is equivalent to maximising $P(X|Y)$, and unstable multiplications become stable sums. From this:

$$h(\mathbf{x}) = \arg \max_k P(Y = k) \prod_{i=1}^d p_{ki}^{x_i}$$

to this:

$$h(\mathbf{x}) = \arg \max_k \left(\log P(Y = k) + \sum_{i=1}^d x_i \cdot \log p_{ki} \right)$$

This log-transformed rule is numerically stable, extremely fast — it is just a sum — and is the one actually implemented in libraries such as `scikit-learn`.



The Bake-Off: The Right vs. The Wrong Tool

We have now seen a family of generative models (QDA, LDA, GNB), and we know discriminative models such as K-NN and Logistic Regression. Let us run a "bake-off" on a **high-dimensional text classification** problem — spam filtering on the SMS Spam Collection — to see which of them is the right tool for the job.

The competitors

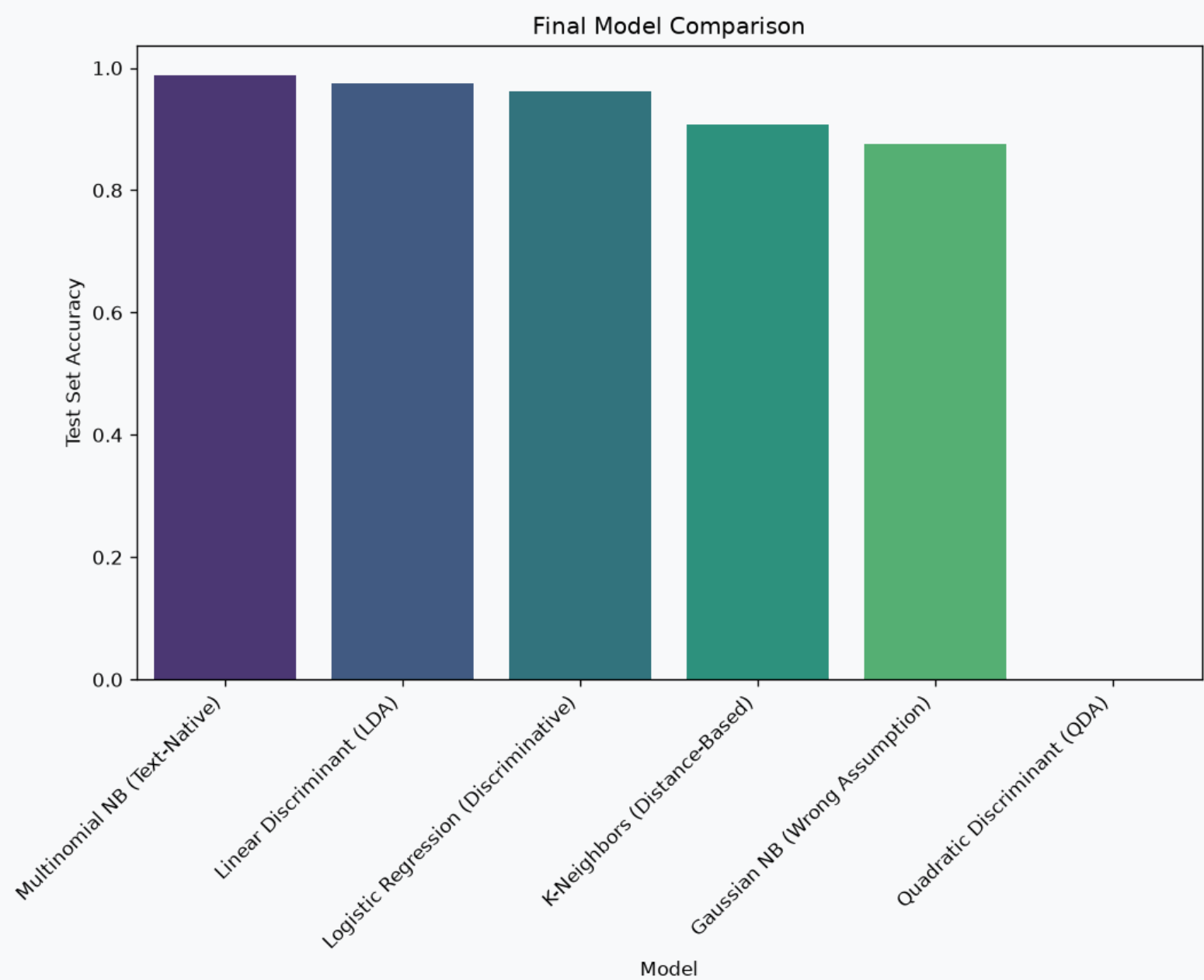
- **Text-native:** `MultinomialNB` (generative, for counts) and `LogisticRegression` (discriminative, works well with high-dimensional text).
- **The "wrong" models:** `KNeighborsClassifier` (distance-based, suffers from the curse of dimensionality), `GaussianNB` (assumes Gaussian features, not counts), `LinearDiscriminantAnalysis` (needs one full covariance matrix), `QuadraticDiscriminantAnalysis` (needs one per class).

The workflow

- Load and clean the data, then make a single train/test split.
- Build a `Pipeline` per model: `CountVectorizer` for MNB, `TfidfVectorizer` for the others.
- The Gaussian models cannot work with the sparse matrix a vectoriser returns, so a custom transformer densifies it — which is itself a sign of the mismatch.
- Fit each model, record accuracy and training time, and catch the errors of the models that fail.



Laboratory: Comparison of Models



Model	Test accuracy	Time (sec)
Multinomial NB (text-native)	98.74%	0.04
Linear Discriminant (LDA)	97.49%	63.63
Logistic Regression (discriminative)	96.23%	0.05
K-Neighbors (distance-based)	90.79%	0.12
Gaussian NB (wrong assumption)	87.50%	0.35
Quadratic Discriminant (QDA)	failed to fit	31.97

98.74%

Multinomial NB

The winner: accurate, and fitted in 0.04 seconds

63.63

LDA time (sec)

Accurate, and more than a thousand times slower

6 889

Features vs 3 377 samples

Why QDA's covariance matrix is not full rank, and the fit fails



The Right Tool for the Job

The bake-off tells a very clear story:

01

The winners

`MultinomialNB` and `LogisticRegression` are designed for high-dimensional, sparse text data, and performed excellently.

03

Curse of dimensionality

`KNeighborsClassifier` was inaccurate: with thousands of features, the distance between two messages is meaningless.

The final lesson: choose a model whose assumptions and parameter count match your data. For high-dimensional, sparse count data like text, **Multinomial Naive Bayes** and **Logistic Regression** are the correct, industry-standard tools.

02

Wrong assumption

`GaussianNB` ran, but poorly: our data are counts, mostly zeros, and nothing like a bell curve.

04

Too many parameters

`LDA` was accurate but took a full minute; `QDA` could not be fitted at all on 6889 features.



Conclusions and Next Steps



We Have Explored:

Generative classifiers

The MAP principle

The "ideal" solution for discrete data

The "ideal" solution for continuous data (QDA)

The LDA simplification and its limits

The naive assumption

Gaussian Naive Bayes

Multinomial Naive Bayes

- In the next lectures we look at data representation and clustering.

References

- Naïve Bayes classifier: en.wikipedia.org/wiki/Naive_Bayes_classifier
- LDA and QDA in scikit-learn: scikit-learn.org/stable/modules/lda_qda.html
- Section 4.4 of [1]

[1] James, Gareth Gareth Michael. *An introduction to statistical learning: with applications in Python*, 2023. statlearning.com

- Lecture notes: [Chapter 16 — Generative classifiers](#)

